🔧 CATEGORY 9: Workflows, CI/CD & Terraform Cloud

> **Difficulty:** Intermediate → Advanced | **Topics:** 11 | **Terraform Interview Mastery Series**

Table of Contents

1. Automating Plan/Apply in CI/CD — GitHub Actions, GitLab, Jenkins
2. PR-Based Workflows — Atlantis vs Terraform Cloud
3. ⚠ Team Workflows — Locking, Approvals, Plan Artifacts
4. Environment Promotion Patterns — dev → staging → prod
5. Terraform Cloud — Remote Runs, VCS Integration, Workspaces
6. ⚠ Sentinel Policies — Policy as Code, Enforcement Levels
7. Useful Flags — `-auto-approve`, `-refresh=false`, `-parallelism`
8. ➕ Agent Pools — Private Network Deployments in TFE/TFC
9. ➕ Run Triggers & Workspace Dependencies — Multi-Stack TFC Architectures
10. ➕ Cost Estimation in TFC — How It Works, Limitations
11. ➕ `removed` Block (Terraform 1.7+) — Clean Resource Removal from State

---

Topic 62: Automating Plan/Apply in CI/CD — GitHub Actions, GitLab, Jenkins

---

🔵 What It Is (Simple Terms)

CI/CD automation for Terraform replaces manual CLI commands with automated pipelines that run `terraform plan` on every PR and `terraform apply` on merge to main. This enforces review, prevents drift, and ensures every infrastructure change is audited and reproducible.

🔵 The Core CI/CD Pattern

```
Pull Request opened:
  1. terraform fmt -check      ← code style check
  2. terraform validate        ← syntax validation
  3. tflint                    ← linting
  4. tfsec / checkov           ← security scan
  5. terraform plan            ← show what will change
  6. Post plan output as PR comment
  → Developer reviews plan before approval

PR merged to main:
  7. terraform plan -out=tfplan  ← re-plan (conditions may have changed)
  8. [Optional] Human approval gate
```

```
  9. terraform apply tfplan      ← apply the saved plan
  10. Post apply output to Slack/Teams
```

🔵 GitHub Actions — Full Production Workflow

```yaml
# .github/workflows/terraform.yml

name: Terraform

on:
  pull_request:
    branches: [main]
    paths: ['environments/prod/**', 'modules/**']
  push:
    branches: [main]
    paths: ['environments/prod/**', 'modules/**']

permissions:
  id-token:      write   # for OIDC
  contents:      read
  pull-requests: write   # for PR comments

env:
  TF_VERSION:   "1.6.0"
  TF_DIRECTORY: "environments/prod"
  AWS_REGION:   "eu-west-1"

jobs:
  # ── Plan job: runs on every PR ──────────────────────────────────────
  plan:
    name: Terraform Plan
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    defaults:
      run:
        working-directory: ${{ env.TF_DIRECTORY }}

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/TerraformRole-
Prod-Plan
          aws-region: ${{ env.AWS_REGION }}

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
```

```
          terraform_version: ${{ env.TF_VERSION }}
          terraform_wrapper: true   # enables plan output capture

      - name: Terraform Format Check
        id: fmt
        run: terraform fmt -check -recursive
        continue-on-error: true

      - name: Terraform Init
        id: init
        run: terraform init -input=false

      - name: Terraform Validate
        id: validate
        run: terraform validate

      - name: Terraform Plan
        id: plan
        run: |
          terraform plan \
            -input=false \
            -out=tfplan \
            -detailed-exitcode \
            2>&1 | tee plan-output.txt
        continue-on-error: true   # exitcode 2 = changes (not an error)

      # Post plan output as PR comment
      - name: Comment Plan on PR
        uses: actions/github-script@v7
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            const fs = require('fs');
            const planOutput = fs.readFileSync(
              '${{ env.TF_DIRECTORY }}/plan-output.txt', 'utf8'
            );

            // Truncate if too long (GitHub comment limit)
            const maxLength = 60000;
            const truncated = planOutput.length > maxLength
              ? planOutput.substring(0, maxLength) + '\n... (truncated)'
              : planOutput;

            const comment = `## 🔧 Terraform Plan — \`prod\`

            #### 📋 Format: \`${{ steps.fmt.outcome }}\`
            #### ✅ Validate: \`${{ steps.validate.outcome }}\`
            #### 📊 Plan: \`${{ steps.plan.outcome }}\`

            <details><summary>Show Plan</summary>

            \`\`\`hcl
            ${truncated}
            \`\`\`
```

```
            </details>

            *Workflow: \`${{ github.workflow }}\` | Run: \`${{
github.run_id }}\`*`;

            // Find and update existing comment, or create new one
            const { data: comments } = await
github.rest.issues.listComments({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
            });

            const existing = comments.find(c =>
              c.body.includes('Terraform Plan') && c.user.login ===
'github-actions[bot]'
            );

            if (existing) {
              await github.rest.issues.updateComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                comment_id: existing.id,
                body: comment,
              });
            } else {
              await github.rest.issues.createComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                issue_number: context.issue.number,
                body: comment,
              });
            }

      # Upload plan file as artifact for use in apply job
      - name: Upload Plan Artifact
        uses: actions/upload-artifact@v4
        with:
          name: tfplan-${{ github.sha }}
          path: ${{ env.TF_DIRECTORY }}/tfplan
          retention-days: 1

      - name: Plan Status Check
        if: steps.plan.outputs.exitcode == '1'
        run: |
          echo "Terraform plan failed!"
          exit 1

  # ── Apply job: runs on merge to main ──────────────────────────────
  apply:
    name: Terraform Apply
    runs-on: ubuntu-latest
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

```yaml
    environment: production   # requires manual approval in GitHub
    defaults:
      run:
        working-directory: ${{ env.TF_DIRECTORY }}

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/TerraformRole-
Prod-Apply
          aws-region: ${{ env.AWS_REGION }}

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      - name: Terraform Init
        run: terraform init -input=false

      - name: Terraform Apply
        run: terraform apply -auto-approve -input=false

      - name: Notify Slack on Success
        if: success()
        uses: slackapi/slack-github-action@v1
        with:
          payload: |
            {
              "text": "✅ Terraform apply succeeded in *prod*\nCommit: ${{
github.sha }}\nAuthor: ${{ github.actor }}"
            }
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}

      - name: Notify Slack on Failure
        if: failure()
        uses: slackapi/slack-github-action@v1
        with:
          payload: |
            {
              "text": "❌ Terraform apply FAILED in *prod*\nCommit: ${{
github.sha }}\nRun: ${{ github.server_url }}/${{ github.repository
}}/actions/runs/${{ github.run_id }}"
            }
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
```

🔵 GitLab CI — Pipeline

```
# .gitlab-ci.yml

stages:
  - validate
  - plan
  - apply

variables:
  TF_VERSION: "1.6.0"
  TF_DIRECTORY: "environments/prod"
  AWS_DEFAULT_REGION: "eu-west-1"

.terraform-base:
  image: hashicorp/terraform:${TF_VERSION}
  id_tokens:
    AWS_OIDC_TOKEN:
      aud: sts.amazonaws.com
  before_script:
    - |
      CREDENTIALS=$(aws sts assume-role-with-web-identity \
        --role-arn "${TERRAFORM_ROLE_ARN}" \
        --role-session-name "gitlab-${CI_JOB_ID}" \
        --web-identity-token "${AWS_OIDC_TOKEN}" \
        --output json)
      export AWS_ACCESS_KEY_ID=$(echo $CREDENTIALS | jq -r
'.Credentials.AccessKeyId')
      export AWS_SECRET_ACCESS_KEY=$(echo $CREDENTIALS | jq -r
'.Credentials.SecretAccessKey')
      export AWS_SESSION_TOKEN=$(echo $CREDENTIALS | jq -r
'.Credentials.SessionToken')
    - cd ${TF_DIRECTORY}
    - terraform init -input=false

validate:
  extends: .terraform-base
  stage: validate
  script:
    - terraform fmt -check -recursive
    - terraform validate

plan:
  extends: .terraform-base
  stage: plan
  script:
    - terraform plan -out=tfplan -input=false
  artifacts:
    name: "tfplan-${CI_COMMIT_SHA}"
    paths: [${TF_DIRECTORY}/tfplan]
    expire_in: 1 hour   # plan artifact only valid briefly
  only: [merge_requests, main]
```

```yaml
apply:
  extends: .terraform-base
  stage: apply
  script:
    - terraform apply -auto-approve tfplan
  dependencies: [plan]
  when: manual        # requires manual trigger in GitLab UI
  only: [main]
  environment:
    name: production
```

🔵 Jenkins Pipeline

```
// Jenkinsfile

pipeline {
    agent { label 'terraform' }

    parameters {
        booleanParam(name: 'AUTO_APPROVE', defaultValue: false,
                     description: 'Skip manual approval for apply')
    }

    environment {
        TF_VERSION   = '1.6.0'
        TF_DIRECTORY = 'environments/prod'
        AWS_REGION   = 'eu-west-1'
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }

        stage('Terraform Init') {
            steps {
                dir(TF_DIRECTORY) {
                    withCredentials([[$class:
'AmazonWebServicesCredentials',
                                      credentialsId: 'terraform-aws-
role']]) {
                        sh 'terraform init -input=false'
                    }
                }
            }
        }

        stage('Terraform Plan') {
            steps {
```

```
                dir(TF_DIRECTORY) {
                    withCredentials([[$class:
'AmazonWebServicesCredentials',
                                      credentialsId: 'terraform-aws-
role']]) {
                        sh '''
                            terraform plan \
                              -input=false \
                              -out=tfplan \
                              -detailed-exitcode || [ $? -eq 2 ]
                        '''
                    }
                }
            }
            post {
                always {
                    archiveArtifacts artifacts: "${TF_DIRECTORY}/tfplan"
                }
            }
        }

        stage('Approval') {
            when {
                not { expression { params.AUTO_APPROVE } }
                branch 'main'
            }
            steps {
                timeout(time: 24, unit: 'HOURS') {
                    input message: 'Review the plan and approve to apply',
                          ok: 'Apply'
                }
            }
        }

        stage('Terraform Apply') {
            when { branch 'main' }
            steps {
                dir(TF_DIRECTORY) {
                    withCredentials([[$class:
'AmazonWebServicesCredentials',
                                      credentialsId: 'terraform-aws-
role']]) {
                        sh 'terraform apply -auto-approve tfplan'
                    }
                }
            }
        }
    }

    post {
        success {
            slackSend channel: '#infra-deploys',
                      color: 'good',
                      message: "✅ Terraform apply succeeded:
```

```
${env.JOB_NAME} #${env.BUILD_NUMBER}"
        }
        failure {
            slackSend channel: '#infra-alerts',
                      color: 'danger',
                      message: "❌ Terraform FAILED: ${env.JOB_NAME}
#${env.BUILD_NUMBER}\n${env.BUILD_URL}"
        }
    }
}
```

🔵 Short Interview Answer

> "A production Terraform CI/CD pipeline has two phases. On pull requests: `fmt -check`, `validate`, `tflint`, security scanning, then `terraform plan` with output posted as a PR comment so reviewers see exactly what will change before approving. On merge to main: `terraform apply` with the apply role (more permissions than the plan role). Plan artifacts are saved and passed to the apply job — this guarantees the apply executes exactly what was reviewed. I use GitHub's `environment` feature with required reviewers as an additional approval gate before production applies. OIDC handles authentication — no stored credentials anywhere."

Topic 63: PR-Based Workflows — Atlantis vs Terraform Cloud

🔵 What It Is (Simple Terms)

Atlantis and **Terraform Cloud (TFC)** are the two dominant tools for enabling PR-based Terraform workflows — where `plan` runs automatically on PR and `apply` is triggered by a PR comment or merge approval.

🔵 Atlantis — Self-Hosted PR Automation

```
What Atlantis does:
  1. You open a PR with Terraform changes
  2. Atlantis detects the PR (webhook from GitHub/GitLab/Bitbucket)
  3. Atlantis automatically runs terraform plan
  4. Posts plan output as a PR comment
  5. Reviewers review the plan in the PR
  6. Reviewer/author comments: "atlantis apply"
  7. Atlantis runs terraform apply, posts result

Architecture:
  - Atlantis server (single Go binary) runs in your infrastructure
```

```
  - Runs inside your VPC — has direct network access to cloud APIs
  - Stores state remotely (S3/GCS backend you configure)
  - Webhooks connect GitHub/GitLab to Atlantis server
```

```yaml
# atlantis.yaml — project configuration

version: 3

projects:
  - name: prod-networking
    dir: environments/prod/networking
    workspace: default
    terraform_version: v1.6.0
    autoplan:
      when_modified: ["*.tf", "*.tfvars", "../../modules/**/*.tf"]
      enabled: true
    apply_requirements:
      - approved          # require at least 1 PR approval
      - mergeable         # PR must be mergeable (no conflicts)
      - undiverged        # PR must be up-to-date with main

  - name: prod-compute
    dir: environments/prod/compute
    workspace: default
    autoplan:
      when_modified: ["*.tf", "../networking/**/*.tf"]
      enabled: true
    apply_requirements:
      - approved
      - mergeable
```

```
# Atlantis PR comment commands
atlantis plan           # re-run plan manually
atlantis plan -d environments/prod/networking   # plan specific directory
atlantis apply          # apply after plan is shown
atlantis apply -d environments/prod/networking  # apply specific project
atlantis unlock         # unlock the workspace if stuck
```

🔵 Atlantis — Pros and Cons

```
Advantages:
  ✅ Runs in your VPC — access to private resources
  ✅ No data leaves your infrastructure (state, plans, logs)
  ✅ Free and open source
  ✅ Familiar PR workflow — everything in GitHub/GitLab
  ✅ Flexible — supports any backend, any provider
```

```
  ✅ Comment-driven — "atlantis apply" is intuitive
  ✅ atlantis.yaml gives fine-grained control per project

Disadvantages:
  ❌ You operate and maintain the Atlantis server
  ❌ Single point of failure (unless you run it HA)
  ❌ Limited UI — just PR comments
  ❌ No native cost estimation
  ❌ No policy enforcement (Sentinel, OPA need separate setup)
  ❌ Concurrency: locks one PR at a time per workspace
  ❌ Secrets management is your responsibility
  ❌ No multi-team RBAC (anyone who can comment can apply)
```

🔵 Terraform Cloud — SaaS PR Automation

```
What TFC does:
  1. VCS integration: GitHub/GitLab repo connected to TFC workspace
  2. PR opened → TFC speculative plan runs automatically
  3. Plan status check appears in GitHub PR checks
  4. PR merged → TFC apply run queued automatically
  5. Apply may require approval (configurable)
  6. Full run history in TFC UI
  7. State stored and encrypted in TFC

Architecture:
  - SaaS platform — HashiCorp operates it
  - Remote execution — runs happen in TFC (unless using agents)
  - Built-in state storage, locking, versioning
  - RBAC, SSO integration
  - Sentinel policies
  - Cost estimation
```

🔵 Atlantis vs Terraform Cloud — Comparison

```
┌──────────────────────────────────────────────────────────────────┐
│                    Atlantis vs Terraform Cloud                   │
│                                                                  │
│  Feature              Atlantis          Terraform Cloud          │
│  ──────────────────────────────────────────────────────────────  │
│  Hosting              Self-hosted       SaaS (HashiCorp)         │
│  Cost                 Free              Free tier + paid plans   │
│  Network access       Your VPC ✅       Agent pool needed ⚠      │
│  State storage        Your S3/GCS       TFC built-in             │
│  Policy enforcement   Manual (OPA)      Sentinel built-in        │
│  Cost estimation      No                Yes (built-in)           │
│  RBAC                 Basic             Full Teams + permissions │
│  Run history          GitHub PR only    Full UI with logs        │
```

```
│  SSO                    GitHub/GitLab     SAML, GitHub, GitLab, etc. │
│  Operations burden      High              Low                        │
│  PR workflow            Comment-driven    VCS integration             │
│  Secrets management     You handle        Variable sets + OIDC       │
│  Compliance             You configure     SOC 2, ISO 27001           │
└────────────────────────────────────────────────────────────────────┘

When to choose Atlantis:
  → Air-gapped/private network — TFC can't reach your resources
  → Strict data residency — state must not leave your infrastructure
  → Cost sensitivity — large team, can't afford TFC Business
  → Existing investment in self-hosted tooling

When to choose Terraform Cloud:
  → Team wants managed solution — no ops burden
  → Need Sentinel policies — enterprise compliance
  → Need cost estimation, team RBAC, SSO out of the box
  → Resources are in public cloud (no private network needed)
```

🔵 Short Interview Answer

> "Atlantis is a self-hosted server that listens for GitHub/GitLab webhooks, automatically runs `terraform plan` on PRs and posts results as comments, then runs `terraform apply` when a reviewer comments `atlantis apply`. It runs inside your VPC so it can reach private resources. Terraform Cloud does the same but as a SaaS — VCS integration triggers speculative plans, results appear as PR status checks, and applies run after approval. TFC adds built-in state management, Sentinel policies, cost estimation, and full team RBAC. Atlantis is the right choice when data must stay in your infrastructure; TFC when you want a managed solution with no operational overhead."

Topic 64: ⚠ Team Workflows — Locking, Approvals, Plan Artifacts

---

🔵 What It Is (Simple Terms)

At team scale, Terraform needs guardrails: only one person applies at a time (locking), multiple people review before production changes (approvals), and the exact plan that was reviewed is what gets applied (plan artifacts). Without these, teams experience state corruption, surprise changes, and audit failures.

🔵 State Locking in Team Workflows

```
Why locking matters in teams:
  ──────────────────────────────────
  Alice starts: terraform apply in prod (lock acquired)
```

```
  Bob tries:    terraform apply in prod (lock failed)
  Bob sees:
    Error: Error acquiring the state lock
    Lock Info:
      ID:   abc123
      Who:  alice@laptop.local
      Created: 10:00:00 UTC

  Bob waits or contacts Alice — no corruption possible

Without locking (no DynamoDB table):
  Alice reads state (serial=47), applies resources A+B
  Bob reads state (serial=47), applies resource C
  Alice writes serial=48 (A+B changes)
  Bob writes serial=48 (C changes) — overwrites Alice's changes!
  Result: A+B exist in AWS but not in state — orphaned resources
```

```
Locking best practices in CI/CD:
  → All applies run through CI/CD — no local applies in production
  → CI/CD jobs are serialized per workspace (queue if another running)
  → GitHub Actions: only one workflow runs per environment at a time
    (concurrency group prevents parallel applies)
```

```yaml
# GitHub Actions — prevent concurrent applies to same environment
jobs:
  apply:
    concurrency:
      group: terraform-apply-prod    # only one job runs at a time
      cancel-in-progress: false      # queue, don't cancel running apply
```

🔵 Approval Gates

```
Three approval models:

1. GitHub Environment with required reviewers (simple, free):
   - Create "production" GitHub Environment
   - Add required reviewers (e.g. 2 senior engineers)
   - Apply workflow references environment: production
   - GitHub pauses the workflow and requests approval
   - Reviewers approve/reject in GitHub UI
   - Apply proceeds only after approval

2. PR review as the approval (Atlantis model):
   - Plan runs on PR
   - Plan output visible in PR comments
   - PR approval = approval of the plan
```

```
   - Merge = apply trigger
   - atlantis.yaml: apply_requirements: [approved, mergeable]

3. Terraform Cloud workspace approval:
   - Workspace configured: "Require approval to apply"
   - Plan runs automatically on push/PR
   - Apply is queued — requires manual confirmation in TFC UI
   - Approval includes who approved and when (audit trail)
```

```
# GitHub: environment with required reviewers
# .github/workflows/terraform.yml

jobs:
  apply:
    environment:
      name: production     # must be created in GitHub Settings >
Environments
      url: ${{ steps.apply.outputs.run_url }}

    # This automatically requests review from configured reviewers
    # The job is BLOCKED until approval is given
```

🔵 Plan Artifacts — The Critical Guarantee

```
The plan artifact problem:
  11:00 → PR opened, plan runs: shows "2 resources to change"
  11:30 → PR approved based on that plan
  11:45 → Someone else merges a different PR
  12:00 → Apply runs from scratch — conditions have changed
          Apply now shows "2 resources to change + 1 to destroy" (!)
          The destruction was NOT what was approved

Solution: Plan artifact workflow
  1. terraform plan -out=tfplan         ← save plan to binary file
  2. Upload tfplan as CI artifact
  3. Human reviews plan output (or PR comment)
  4. terraform apply tfplan             ← apply EXACTLY the saved plan
     (no re-evaluation — locked to that specific plan)

  Guarantees: what was reviewed = what was applied
  Even if config changed between plan and apply
  Even if cloud state changed between plan and apply
```

```
# GitHub Actions: plan artifact pattern
jobs:
  plan:
```

```yaml
    steps:
      - name: Terraform Plan
        run: terraform plan -out=tfplan

      - name: Upload Plan Artifact
        uses: actions/upload-artifact@v4
        with:
          name: tfplan-${{ github.sha }}
          path: environments/prod/tfplan
          retention-days: 1   # expire quickly — plans have time-sensitive
data

  apply:
    needs: [plan]    # must wait for plan job
    environment: production   # requires approval before this job runs

    steps:
      - name: Download Plan Artifact
        uses: actions/download-artifact@v4
        with:
          name: tfplan-${{ github.sha }}
          path: environments/prod/

      - name: Terraform Apply Saved Plan
        run: terraform apply tfplan   # applies EXACTLY the saved plan
```

🔵 Plan Expiry and Safety Windows

```
Plan artifacts have a safety window — how long is a plan valid?

Short window (minutes): Very tight consistency, but approval difficult
Long window (hours):    Comfortable for review, but state may drift

Recommended: 1-4 hours maximum
  → Plans older than this should be re-run before applying
  → Terraform itself doesn't enforce expiry — you enforce via CI/CD

Why plans expire:
  - Cloud state may drift (someone made a manual change)
  - Another Terraform apply in a dependent stack completed
  - AWS API state changed (e.g., certificate validation completed)

Best practice: Apply within the same CI/CD run as plan
  Plan + Approval + Apply in one pipeline run
  Only the human approval step adds real time
  Prevents plan drift entirely
```

🔵 Short Interview Answer

> "Three essential team workflow mechanisms: State locking prevents concurrent applies that corrupt state — DynamoDB handles this for S3 backends. Approvals ensure the right people review changes before production — GitHub Environments with required reviewers, Atlantis `apply_requirements`, or TFC workspace approval. Plan artifacts are the most important: saving `terraform plan -out=tfplan` and applying it with `terraform apply tfplan` guarantees the exact plan that was reviewed is what gets applied — even if configuration or cloud state changed in between. Without plan artifacts, the time gap between PR approval and apply can result in unexpected additional changes being applied."

Topic 65: Environment Promotion Patterns — dev → staging → prod

---

🔵 What It Is (Simple Terms)

Environment promotion is the process of moving infrastructure changes safely through environments — dev first, then staging, then prod — each with appropriate testing and gates, so production changes are validated before they go live.

🔵 The Three Main Promotion Strategies

Strategy 1: Mono-repo with Sequential Pipeline

```
Repository structure:
  environments/
    dev/       ← identical modules, dev-specific config
    staging/   ← identical modules, staging-specific config
    prod/      ← identical modules, prod-specific config
  modules/
    vpc/
    rds/
    eks/

Pipeline flow:
  PR opened → plan all three envs (validate config compiles for all)
  Merge → apply dev
    → automated tests against dev (integration tests, smoke tests)
    → if tests pass: apply staging
    → automated tests against staging
    → human approval gate
    → apply prod

Advantages:
  ✅ Config change applied identically across all environments
  ✅ If it works in staging, same config applies to prod
  ✅ Clear promotion chain — visible in pipeline
```

```
Disadvantages:
  ❌ Environments may legitimately differ (prod has more resources)
  ❌ Prod apply blocked waiting for dev/staging
  ❌ One flaky test blocks all environments
```

```yaml
# GitHub Actions: sequential environment promotion
name: Terraform Promote

on:
  push:
    branches: [main]

jobs:
  deploy-dev:
    uses: ./.github/workflows/terraform-apply.yml
    with:
      environment: dev
      directory: environments/dev

  test-dev:
    needs: [deploy-dev]
    runs-on: ubuntu-latest
    steps:
      - name: Run integration tests against dev
        run: |
          pytest tests/integration/ \
            --env=dev \
            --endpoint=${{ needs.deploy-dev.outputs.api_endpoint }}

  deploy-staging:
    needs: [test-dev]
    uses: ./.github/workflows/terraform-apply.yml
    with:
      environment: staging
      directory: environments/staging

  test-staging:
    needs: [deploy-staging]
    runs-on: ubuntu-latest
    steps:
      - name: Run smoke tests against staging
        run: |
          pytest tests/smoke/ --env=staging

  deploy-prod:
    needs: [test-staging]
    environment: production   # requires manual approval
    uses: ./.github/workflows/terraform-apply.yml
    with:
```

```
        environment: prod
        directory: environments/prod
```

Strategy 2: Branch-per-Environment (GitFlow Style)

```
Branch structure:
  main      → deploys to prod
  staging   → deploys to staging
  develop   → deploys to dev

Promotion flow:
  1. Feature branch: develop changes
  2. PR to develop → applies to dev
  3. PR from develop to staging → applies to staging
  4. Testing in staging → PR from staging to main → applies to prod

Advantages:
  ✅ Branch = environment (clear mental model)
  ✅ Staging branch is always what's running in staging

Disadvantages:
  ❌ Branch divergence — develop, staging, main can differ
  ❌ Merge conflicts when promoting
  ❌ Hard to track "is this change in prod?" across branches
  ❌ Generally falling out of favor for trunk-based development
```

Strategy 3: Tag-Based Promotion

```
Workflow:
  1. Merge to main → deploys to dev automatically
  2. Create release tag: git tag -a v2024.01.15 -m "Release"
  3. Tag push → pipeline applies to staging
  4. After validation: git tag -a v2024.01.15-prod -m "Promote to prod"
  5. -prod tag push → applies to prod with approval gate

Advantages:
  ✅ Explicit promotion artifacts — tags are immutable records
  ✅ Rollback is clear: deploy previous tag
  ✅ Audit trail: when was v2024.01.15 promoted to prod?

Disadvantages:
  ❌ Manual tag creation adds friction
  ❌ Tag management overhead in high-velocity teams
```

🔵 What Changes Between Environments

```
# Same module, different environment configs

# environments/dev/terraform.tfvars
environment        = "dev"
vpc_cidr           = "10.1.0.0/16"
instance_type      = "t3.micro"
min_capacity       = 1
max_capacity       = 2
enable_nat_gateway = false   # save cost in dev
multi_az           = false   # no HA in dev
backup_retention   = 0       # no backups in dev

# environments/prod/terraform.tfvars
environment        = "prod"
vpc_cidr           = "10.0.0.0/16"
instance_type      = "t3.large"
min_capacity       = 3
max_capacity       = 20
enable_nat_gateway = true    # required in prod
multi_az           = true    # HA in prod
backup_retention   = 30      # 30-day backups in prod
```

🔵 Promotion Gates — What Sits Between Environments

```
Gate 1: Between dev and staging
  → Automated integration tests pass
  → No HIGH/CRITICAL security findings in tfsec/checkov
  → Plan looks correct (no unexpected destroys)

Gate 2: Between staging and prod
  → Automated smoke tests pass against staging
  → Load test results meet SLO thresholds
  → Manual review by infrastructure lead
  → Change management ticket approved (in regulated environments)
  → Business hours only (no prod deploys on Friday afternoon)

Rollback plan must exist before prod deploy:
  → Know the previous working commit SHA
  → Verify `terraform plan` on previous SHA shows safe rollback
  → Have runbook ready for manual rollback if pipeline fails
```

🔵 Short Interview Answer

> "Environment promotion moves changes dev → staging → prod with validation gates between each step. The most common pattern in trunk-based development: merge to main triggers dev apply, automated tests run against dev, if they pass staging applies automatically, smoke tests run, then a

> human approval gate protects prod. The key insight is that all environments use the same modules with environment-specific variable files — this ensures what passes staging is structurally identical to what applies to prod. Gates between environments include automated tests, security scan results, and manual approval for prod. Rollback strategy must be decided before the prod deploy begins."

Topic 66: Terraform Cloud — Remote Runs, VCS Integration, Workspaces

---

🔵 What It Is (Simple Terms)

Terraform Cloud is HashiCorp's SaaS platform for running Terraform at team scale. It provides managed state storage, remote plan/apply execution, VCS integration for automatic runs, team access controls, and a web UI for visibility into all infrastructure changes.

🔵 TFC Workspaces — The Core Unit

```
Each TFC workspace:
  - Has its own state file (encrypted, versioned)
  - Maps to one root module configuration
  - Connected to one VCS repo + branch (or CLI-driven)
  - Has its own variable set
  - Has its own run history
  - Has its own team permissions
  - Executes in its own isolated environment

Workspace types:
  VCS-driven: auto-plan on VCS push, apply after approval
  CLI-driven:  plan/apply initiated by terraform CLI
  API-driven:  plan/apply triggered by API call (CI/CD integration)
  No-code:     deploy via TFC UI form (for non-engineers)
```

🔵 VCS Integration Setup

```
# Terraform config to use TFC backend
terraform {
  cloud {
    organization = "mycompany"

    workspaces {
      name = "prod-networking"
      # OR: tags = ["prod", "networking"]
    }
```

```
  }
}

# After terraform init, all plan/apply happens in TFC
# terraform plan → uploads config → TFC runs plan in its infrastructure
# terraform apply → TFC runs apply, returns results to CLI
```

```
VCS integration setup steps:
  1. Connect TFC organization to GitHub/GitLab
     (OAuth integration in TFC Settings > Version Control)

  2. Create workspace: Source = VCS, repo = myorg/infra-repo

  3. Configure:
     Working Directory: environments/prod
     Branch: main (auto-apply on push)
     Trigger paths: environments/prod/**, modules/**

  4. On PR:
     TFC creates "speculative plan" — read-only, shows what would change
     Plan status appears as GitHub PR check
     Merge is blocked until plan check passes (configurable)

  5. On merge to main:
     TFC automatically queues and runs apply
     (or waits for manual approval if workspace requires it)
```

🔵 Remote Execution — What Happens Behind the Scenes

```
When you run terraform plan in TFC:

1. CLI uploads:
   - Terraform config files (.tf, .tfvars)
   - NOT your .terraform directory — TFC downloads providers itself

2. TFC creates a "run" in an isolated container:
   - Downloads Terraform binary (version from workspace config)
   - Downloads providers (from registry or private mirror)
   - Sets environment variables you've configured
   - Reads state from TFC state storage

3. Plan executes in TFC container:
   - Makes API calls to cloud providers (AWS, GCP, etc.)
   - Generates plan
   - Streams output back to your CLI in real-time

4. Plan result stored in TFC
   - Can be reviewed in TFC UI
```

```
    - Apply can be triggered from UI or CLI or auto-triggered on merge

5. After apply:
   - State updated in TFC storage
   - Run stored in history with full logs

Security implications:
  - Your cloud credentials are stored as TFC workspace variables
  - TFC containers have outbound internet access
  - For private resources: agent pools required (Topic 69)
```

🔵 TFC Variables — Types and Priority

```
TFC supports three variable types per workspace:

1. Terraform variables (HCL values — equivalent to tfvars)
   - Appear as terraform variables at plan time
   - Can be marked sensitive (encrypted in TFC, redacted in logs)
   - Override: config defaults < workspace variables < variable sets

2. Environment variables (KEY=VALUE in shell)
   - Set for the plan/apply execution environment
   - Used for provider auth: AWS_ACCESS_KEY_ID, TFE_TOKEN, etc.
   - Use OIDC instead of static AWS keys (with AWS provider OIDC support)

3. Variable sets (shared across workspaces)
   - Define once, apply to many workspaces
   - Examples: company-wide AWS credentials, global tags
   - Can be applied to: all workspaces, specific workspaces, by tag

variable_sets:
  "global-aws-auth"     → applied to all workspaces
    AWS_ROLE_ARN = arn:aws:iam::123456789012:role/TerraformRole
    TFC_AWS_PROVIDER_AUTH = true
    TFC_AWS_RUN_ROLE_ARN = arn:aws:iam::...

  "prod-config"         → applied to prod-* workspaces
    environment = "prod"
    vpc_cidr = "10.0.0.0/16"
```

🔵 TFC Run Lifecycle

```
Run States:
  pending         → queued, waiting for workspace to be available
  plan_queued     → waiting for run capacity
  planning        → terraform plan executing
  cost_estimating → cost estimation running (if enabled)
```

```
  policy_checking → Sentinel policies evaluating
  policy_override → policy soft-fail, waiting for override
  apply_queued    → waiting to apply (if auto-apply disabled)
  applying        → terraform apply executing
  applied         → complete ✅
  discarded       → run cancelled by user
  errored         → plan or apply failed ❌
```

🔵 Short Interview Answer

> "Terraform Cloud's core unit is the workspace — each has its own state, variable set, team permissions, and run history. VCS integration connects a workspace to a repo branch: pushes trigger speculative plans (shown as PR status checks), merges trigger applies. Remote execution means plans/applies happen in TFC-managed containers — your credentials are stored as workspace variables, TFC makes the API calls. Variable sets let you define shared config (AWS credentials, environment tags) once and apply to multiple workspaces. The run lifecycle goes: pending → planning → cost estimation → Sentinel policy check → apply (manual or auto) → applied. For resources in private networks, agent pools bridge TFC to your VPC."

Topic 67: ⚠️ Sentinel Policies — Policy as Code, Enforcement Levels

---

🔵 What It Is (Simple Terms)

Sentinel is HashiCorp's **policy-as-code framework** embedded in Terraform Cloud/Enterprise. It lets you write policies in the Sentinel language that are automatically evaluated on every Terraform run — preventing non-compliant infrastructure from being applied.

🔵 Why Sentinel Exists

```
Problem: Terraform is powerful — teams can create insecure or expensive
infra

Without Sentinel:
  → Developer creates S3 bucket with public read access
  → terraform apply succeeds — no one stopped it
  → Security incident from misconfigured bucket

  → Developer creates a db.r5.8xlarge RDS instance
  → terraform apply succeeds — $5000/month surprise cost

  → Team uses unapproved community Terraform module
  → No visibility, no control
```

```
With Sentinel:
  → S3 bucket public? → Policy HARD_FAIL → apply blocked
  → Instance class not in approved list? → Policy SOFT_FAIL → requires
override
  → Module not from private registry? → Policy ADVISORY → warning emitted
```

🔵 The Three Enforcement Levels

```
ADVISORY: Emit a warning, never block
  → Policy failed but the run continues
  → Used for: informational policies, gradually rolling out a new rule
  → Good for: "hey, you forgot to add a cost_center tag"

SOFT_MANDATORY: Block apply, but authorized users can override
  → Run shows policy failure and pauses
  → Users with "Manage Policy Overrides" permission can approve to proceed
  → Used for: "unusual but sometimes valid" configurations
  → Good for: instance types outside approved list (but CISO can approve)

HARD_MANDATORY: Block apply, NO override possible
  → Run fails — no one can override, period
  → Used for: absolute security requirements
  → Good for: no S3 public access, no unencrypted databases, no internet-
exposed SSH
```

🔵 Writing Sentinel Policies — Syntax

```
# sentinel/require-tags.sentinel

import "tfplan/v2" as tfplan

# Main rule: all managed resources must have required tags
main = rule {
    all_resources_have_required_tags
}

# Helper: all resources being created/modified have required tags
all_resources_have_required_tags = rule {
    all tfplan.resource_changes as _, resource_changes {
        all resource_changes as _, rc {
            rc.change.actions contains "no-op" or
            has_required_tags(rc.change.after)
        }
    }
}
```

```
# Helper: check for required tags
has_required_tags = func(resource) {
    tags = resource.tags else {}
    all ["Environment", "Team", "CostCenter", "ManagedBy"] as required_tag
{
        tags[required_tag] is defined and
        tags[required_tag] is not ""
    }
}
```

```
# sentinel/restrict-instance-types.sentinel

import "tfplan/v2" as tfplan

# Approved EC2 instance types
approved_instance_types = [
    "t3.micro", "t3.small", "t3.medium", "t3.large",
    "m5.large", "m5.xlarge", "m5.2xlarge",
    "c5.large", "c5.xlarge",
]

# Find all EC2 instances in the plan
ec2_instances = filter tfplan.resource_changes as _, rc {
    rc.type is "aws_instance" and
    rc.change.actions contains "create" or rc.change.actions contains
"update"
}

# Check all instances use approved types
instances_use_approved_types = rule {
    all ec2_instances as _, instance {
        instance.change.after.instance_type in approved_instance_types
    }
}

main = rule {
    instances_use_approved_types
}
```

```
# sentinel/no-public-s3.sentinel

import "tfplan/v2" as tfplan

# Find all S3 bucket public access blocks being created/modified
s3_buckets_public_access = filter tfplan.resource_changes as _, rc {
    rc.type is "aws_s3_bucket_public_access_block" and
    (rc.change.actions contains "create" or rc.change.actions contains
"update")
}
```

```
# All buckets must block public access
buckets_block_public_access = rule {
    all s3_buckets_public_access as _, pab {
        pab.change.after.block_public_acls      is true and
        pab.change.after.block_public_policy     is true and
        pab.change.after.ignore_public_acls      is true and
        pab.change.after.restrict_public_buckets is true
    }
}

main = rule when tfplan.phase is "apply" {
    buckets_block_public_access
}
```

🔵 Policy Sets — Organizing and Applying Policies

```
# Policy set: group policies and apply to workspaces

# In TFC UI or via API:
# Policy Set: "security-baseline"
#   Policies:
#     - require-tags.sentinel         (SOFT_MANDATORY)
#     - no-public-s3.sentinel         (HARD_MANDATORY)
#     - restrict-instance-types.sentinel (SOFT_MANDATORY)
#   Applied to: all workspaces with tag "env:prod"

# TFC API to create policy set:
resource "tfe_policy_set" "security_baseline" {
  name          = "security-baseline"
  description   = "Core security policies for all production workspaces"
  organization  = "mycompany"
  workspace_ids = [tfe_workspace.prod_networking.id,
tfe_workspace.prod_compute.id]
  # OR: global = true  # applies to ALL workspaces

  policy_ids = [
    tfe_sentinel_policy.require_tags.id,
    tfe_sentinel_policy.no_public_s3.id,
    tfe_sentinel_policy.restrict_instance_types.id,
  ]
}

resource "tfe_sentinel_policy" "no_public_s3" {
  name         = "no-public-s3"
  description  = "S3 buckets must block all public access"
  organization = "mycompany"
  policy       = file("sentinel/no-public-s3.sentinel")
  enforce_mode = "hard-mandatory"   # cannot be overridden
}
```

🔵 Sentinel Run — What It Looks Like

```
Run timeline:
  1. plan       → terraform plan executes
  2. cost_est   → cost estimation runs (if enabled)
  3. sentinel   → each policy evaluates against the plan

Sentinel evaluation output:

   ┌──────────────────────────────────────────────────────┐
   │ Policy Check: security-baseline                      │
   │                                                      │
   │ ✅  require-tags.sentinel    PASS   (soft-mandatory)  │
   │ ❌  no-public-s3.sentinel    FAIL   (hard-mandatory)  │
   │    S3 bucket 'aws_s3_bucket.assets' missing public   │
   │    access block configuration                        │
   │                                                      │
   │ Result: HARD_FAIL — Apply is blocked                 │
   │ Required action: Fix the S3 bucket configuration     │
   └──────────────────────────────────────────────────────┘

For SOFT_MANDATORY failure:
  Result: SOFT_FAIL — Apply requires override
  Override available to: Policy Managers, Organization Owners
  Override reason required: [text input]
```

🔵 Short Interview Answer

> "Sentinel is HashiCorp's policy-as-code framework that evaluates Terraform plans before apply. Policies are written in the Sentinel language and can access the full plan — resources being created, their attributes, the cost estimate. There are three enforcement levels: `advisory` (warn only), `soft-mandatory` (block but authorized users can override with a reason), and `hard-mandatory` (block with no override — absolute requirement). Policies are grouped into policy sets applied to workspaces. Common uses: require specific tags on all resources, restrict instance types to approved list, block S3 public access, enforce encryption on all databases. Sentinel gives platform teams enforceable standards that no developer can bypass — even with full Terraform permissions."

Topic 68: Useful Flags — `-auto-approve`, `-refresh=false`, `-parallelism`

🔵 What It Is (Simple Terms)

Terraform's CLI flags let you control plan and apply behavior — skipping confirmation prompts, controlling whether state is refreshed, tuning parallelism, and more. Knowing when to use each flag (and when NOT to) separates experienced operators from beginners.

🔵 -auto-approve

```
# Skip the interactive "Do you want to perform these actions?" prompt
terraform apply -auto-approve

# When to use:
✅ CI/CD pipelines — automated applies cannot wait for input
✅ Applying a saved plan file (no interactive confirmation anyway)
✅ Non-production environments where speed matters

# When NOT to use:
❌ Manually running apply in production
❌ Any context where a human should review the plan first
❌ Combined with -target (double danger)

# Safer pattern in CI/CD: always apply a saved plan
terraform plan -out=tfplan          # review this
terraform apply -auto-approve tfplan  # applies exactly what was reviewed
```

🔵 -refresh=false

```
# Skip the provider API calls to refresh current state
# Uses last-known state values instead of querying cloud APIs
terraform plan -refresh=false
terraform apply -refresh=false

# What it skips:
# - Calling DescribeInstances, DescribeVpcs, etc. for every managed
resource
# - The network round-trips to cloud APIs

# When to use:
✅ Debugging a specific config change when you're certain no manual
changes occurred
✅ Speeding up plan in large infrastructures (100s of resources)
✅ When provider API is throttling and causing timeouts
✅ Air-gapped environments where API access is limited

# When NOT to use:
❌ Any time drift may have occurred (production operations)
❌ Before a significant apply (could miss deleted resources)
❌ After an incident (state may not reflect current reality)

# Speed comparison:
```

```
# Normal plan (200 resources): ~3-5 minutes (200 API calls)
# -refresh=false (200 resources): ~30 seconds (0 API calls)
```

🔵 -parallelism=N

```
# Default: 10 concurrent resource operations
terraform apply -parallelism=10

# Increase for faster applies
terraform apply -parallelism=20   # double the concurrency

# Decrease for throttling/rate-limit issues
terraform apply -parallelism=2    # slow down to avoid API throttling

# When to INCREASE:
✅ Large infrastructure with many independent resources
✅ Fast network, low-latency provider APIs
✅ Resources don't have dependencies (can all create in parallel)

# When to DECREASE:
✅ AWS API throttling errors (TooManyRequestsException)
✅ Provider rate limiting
✅ Database connection limits (creating many RDS instances)
✅ Debugging — easier to follow with sequential operations

# Important: -parallelism doesn't affect DEPENDENT resources
# If resource B depends on resource A (explicitly or implicitly):
# A must complete before B starts — regardless of parallelism setting
# Parallelism only affects INDEPENDENT resource operations
```

🔵 -target

```
# Scope an apply to specific resources and their dependencies
terraform plan -target=aws_instance.web
terraform apply -target=aws_instance.web
terraform apply -target=module.networking

# Also works with multiple targets:
terraform apply \
  -target=aws_security_group.app \
  -target=aws_instance.web

# When to use:
✅ Emergency: update a single resource without full apply
✅ Bootstrap: chicken-and-egg (IAM role needed before full apply)
✅ Testing: applying a single resource in development
```

```
# When NOT to use:
❌ Routine operations — always do full apply
❌ Ongoing: using -target regularly = sign your config is wrong
❌ Without doing a full apply afterward (leaves state inconsistent)

# The danger:
resource "aws_instance" "web" {
  security_groups = [aws_security_group.web.id]
}

# If you: terraform apply -target=aws_security_group.web
# Security group is created, state is updated
# But aws_instance.web is NOT updated — still references the SG ID
# Next full plan: may show unexpected changes due to partial state
```

🔵 -detailed-exitcode

```
# Change exit code behavior for scripting
terraform plan -detailed-exitcode

# Exit codes:
# 0 = success, no changes (nothing to apply)
# 1 = error
# 2 = success, changes present (plan has differences)

# In CI/CD scripts:
terraform plan -detailed-exitcode -out=tfplan
PLAN_EXIT=$?

if [ $PLAN_EXIT -eq 0 ]; then
  echo "No changes needed"
elif [ $PLAN_EXIT -eq 2 ]; then
  echo "Changes detected — applying..."
  terraform apply tfplan
elif [ $PLAN_EXIT -eq 1 ]; then
  echo "Plan failed!"
  exit 1
fi
```

🔵 -replace

```
# Force a specific resource to be destroyed and recreated
terraform apply -replace=aws_instance.web
terraform apply -replace="aws_instance.web[\"api\"]"

# Equivalent to old terraform taint (deprecated)
```

```
# When to use:
✅ Resource is in a broken state but plan shows no changes
✅ Forcing rolling replacement of an EC2 instance
✅ Testing destruction/recreation of a specific resource

# Important: Shows in plan as -/+ (destroy + create)
# Review the plan carefully — replacement may affect dependent resources
```

🔵 TF_LOG — Debug Logging

```
# Enable debug logging
export TF_LOG=DEBUG     # very verbose — all HTTP requests/responses
export TF_LOG=INFO      # provider info messages
export TF_LOG=WARN      # warnings only
export TF_LOG=ERROR     # errors only
export TF_LOG=TRACE     # even more verbose than DEBUG

# Save logs to file
export TF_LOG=DEBUG
export TF_LOG_PATH=./terraform-debug.log
terraform apply

# Specific provider logging
export TF_LOG_PROVIDER=DEBUG   # only provider plugin logs

# Use cases:
# DEBUG: diagnosing provider API issues, why a resource keeps recreating
# INFO:  understanding what Terraform is doing step by step
# WARN:  checking for deprecation warnings in providers
```

🔵 TF_CLI_ARGS

```
# Set default flags for all commands
export TF_CLI_ARGS="-no-color"    # no ANSI colors (useful in CI)
export TF_CLI_ARGS_plan="-compact-warnings"  # for plan command only
export TF_CLI_ARGS_apply="-auto-approve"    # for apply command only

# In CI/CD:
export TF_CLI_ARGS="-no-color -input=false"
terraform plan   # automatically gets -no-color -input=false
```

🔵 Short Interview Answer

> "`-auto-approve` skips the interactive confirmation — required for CI/CD, dangerous for manual production applies. `-refresh=false` skips provider API calls to check current state — massively speeds up plans but blinds Terraform to drift. Use it only when you're certain no manual changes occurred. `-parallelism=N` controls concurrent resource operations — increase for speed on large independent infrastructure, decrease for API throttling. Critically, `-parallelism` never affects dependent resources — they still execute in dependency order. `-detailed-exitcode` is essential for CI scripting: exit 0 means no changes, exit 2 means changes present, exit 1 means error. `-replace` forces destroy+recreate of a specific resource without destroying dependents."

Topic 69: ➕ Agent Pools — Private Network Deployments in TFE/TFC

🔵 What It Is (Simple Terms)

By default, Terraform Cloud runs plans and applies in HashiCorp-managed containers with outbound internet access. **Agent pools** allow TFC to run workloads inside **your own private network** — enabling Terraform to reach resources that aren't internet-accessible (private RDS instances, on-prem systems, internal Kubernetes clusters).

🔵 The Problem Agents Solve

```
TFC default execution:
  TFC Container (HashiCorp's network)
    → makes API calls to AWS (public endpoint) ✅
    → makes API calls to GCP ✅
    → tries to reach private RDS at 10.0.5.20 ❌ (not routable)
    → tries to reach on-prem VMware vCenter ❌
    → tries to reach Kubernetes API (private endpoint) ❌

With agent pools:
  TFC Agent (running IN your VPC)
    → makes API calls to AWS ✅
    → reaches private RDS at 10.0.5.20 ✅ (same VPC)
    → reaches on-prem VMware via Direct Connect ✅
    → reaches private Kubernetes API ✅

  Flow:
    TFC signals agent: "new job ready"
    Agent polls TFC API (outbound HTTPS from your network)
    Agent downloads job details + Terraform config
    Agent runs plan/apply with network access to private resources
    Agent reports results back to TFC
```

🔵 Agent Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│  Your VPC                                                        │
│                                                                  │
│   ┌──────────────────┐        HTTPS (outbound)     ┌──────────────┐  │
│   │  TFC Agent       │ ←──────────────────────────→ │  app.terraform.io │
│   │  (container/VM)  │        polls for jobs        │  (TFC SaaS)  │  │
│   │                  │        downloads config      │              │  │
│   │  Has access to:  │        reports results       │  Sends jobs  │  │
│   │  - Private RDS   │                              │  to agent    │  │
│   │  - Private EKS   │                              │              │  │
│   │  - On-prem hosts │                              └──────────────┘  │
│   └──────────────────┘                                                 │
│          ↕ private network                                             │
│   ┌──────────────────┐                                                 │
│   │  Private RDS     │                                                 │
│   │  Private EKS     │                                                 │
│   │  Internal APIs   │                                                 │
│   └──────────────────┘                                                 │
└──────────────────────────────────────────────────────────────────┘
```

🔵 Deploying TFC Agents

```
# Run agent as ECS task in your VPC
resource "aws_ecs_task_definition" "tfc_agent" {
  family                   = "tfc-agent"
  requires_compatibilities = ["FARGATE"]
  network_mode             = "awsvpc"
  cpu                      = 1024
  memory                   = 2048

  container_definitions = jsonencode([{
    name  = "tfc-agent"
    image = "hashicorp/tfc-agent:latest"

    environment = [
      {
        name  = "TFC_AGENT_TOKEN"
        # ← agent token from TFC workspace settings
        # Store in Secrets Manager, inject here
        value =
data.aws_secretsmanager_secret_version.tfc_token.secret_string
      },
      {
        name  = "TFC_AGENT_NAME"
        value = "prod-vpc-
agent-${aws_ecs_task_definition.tfc_agent.revision}"
      },
```

```
      {
        name  = "TFC_ADDRESS"
        value = "https://app.terraform.io"
      }
    ]

    logConfiguration = {
      logDriver = "awslogs"
      options = {
        "awslogs-group"  = "/ecs/tfc-agent"
        "awslogs-region" = "eu-west-1"
      }
    }
  }])
}

resource "aws_ecs_service" "tfc_agent" {
  name            = "tfc-agent"
  cluster         = aws_ecs_cluster.platform.id
  task_definition = aws_ecs_task_definition.tfc_agent.arn
  desired_count   = 2    # run 2 agents for redundancy + concurrency

  network_configuration {
    subnets         = aws_subnet.private[*].id    # private subnets — no
public IP
    security_groups = [aws_security_group.tfc_agent.id]
  }
}

# Security group: only outbound HTTPS needed
resource "aws_security_group" "tfc_agent" {
  name   = "tfc-agent"
  vpc_id = aws_vpc.main.id

  egress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]   # outbound HTTPS to TFC + cloud APIs
  }
  # No inbound rules needed — agent polls TFC (outbound only)
}
```

🔵 Configuring Workspace to Use Agent Pool

```
TFC UI:
  Workspace Settings → General → Execution Mode

Options:
  Remote (default): runs in TFC-managed containers
```

```
  Local:            runs on your CLI machine (not recommended for teams)
  Agent:            runs on your agent pool

Select "Agent" → Select agent pool → Save

Now all runs for this workspace use your agent
```

🔵 Short Interview Answer

> "TFC Agent pools let Terraform Cloud execute plans and applies inside your private network. The agent runs in your VPC (as an ECS task, Kubernetes pod, or VM), polls TFC via outbound HTTPS for available jobs, downloads the config, runs plan/apply with direct access to private resources like RDS, private EKS, or on-prem systems, then reports results back to TFC. No inbound firewall rules are needed — only outbound HTTPS on port 443. Run multiple agents for redundancy and parallel execution. The agent's IAM role or instance profile provides the cloud credentials — not stored in TFC variables. This is the correct architecture when you can't expose infrastructure to the public internet but still want TFC's UI, state management, and policy features."

Topic 70: ➕ Run Triggers & Workspace Dependencies — Multi-Stack TFC Architectures

---

🔵 What It Is (Simple Terms)

Run triggers in Terraform Cloud automatically queue a plan in one workspace when another workspace successfully applies. This models infrastructure dependencies — when the networking stack changes, the compute stack that depends on it should automatically re-plan.

🔵 Why Run Triggers Are Needed

```
Without run triggers (manual dependency management):
  Networking team applies changes to VPC
  Compute workspace uses networking outputs via terraform_remote_state
  But compute workspace doesn't know networking changed
  → Compute team must manually trigger a plan to pick up new values
  → Easy to forget → drift between dependent stacks

With run triggers:
  Networking workspace applies successfully
  → TFC automatically queues plan in compute workspace
  → Compute workspace picks up latest networking outputs
  → If compute plan shows changes: queued for approval/auto-apply
  → Dependency chain is automated
```

🔵 Configuring Run Triggers

```
TFC UI:
  Workspace (compute) → Settings → Run Triggers
  → Add workspace: "prod-networking"

Now: when prod-networking applies successfully
  → prod-compute is automatically queued for a new plan

TFC API / Terraform:
```

```
# Configure run trigger via TFC provider
resource "tfe_workspace" "compute" {
  name         = "prod-compute"
  organization = "mycompany"
}

resource "tfe_workspace" "networking" {
  name         = "prod-networking"
  organization = "mycompany"
}

resource "tfe_run_trigger" "compute_depends_on_networking" {
  workspace_id  = tfe_workspace.compute.id        # downstream workspace
  sourceable_id = tfe_workspace.networking.id     # upstream workspace
  # When networking applies → compute is triggered
}
```

🔵 Multi-Stack Dependency Chains

```
Example: Full infrastructure dependency chain

  prod-networking (VPC, subnets)
       ↓ run trigger
  prod-security (IAM, security groups)  +  prod-data (RDS, Redis)
       ↓ run trigger                          ↓ run trigger
  prod-compute (EKS cluster, ASG)
       ↓ run trigger
  prod-applications (services, deployments)

Configure:
  prod-security trigger: prod-networking
  prod-data trigger: prod-networking
  prod-compute trigger: prod-security + prod-data (both)
  prod-applications trigger: prod-compute
```

```
Behavior:
  When you apply prod-networking:
  → prod-security and prod-data both queue plans (parallel)
  → After both apply: prod-compute queues plan
  → After compute applies: prod-applications queues plan
```

🔵 Run Trigger Behavior Details

```
Important behaviors:

1. Apply triggers, not plan
   → Trigger fires after SUCCESSFUL APPLY (not just plan)
   → If upstream plan runs but apply is manual: no trigger yet

2. Only queues a plan — doesn't force apply
   → Downstream workspace still goes through its normal flow
   → If workspace requires approval: apply still needs human approval

3. Trigger even if "no changes"
   → Upstream apply with 0 changes still triggers downstream
   → Downstream plan may also show 0 changes — that's fine

4. One trigger per workspace pair
   → Can have multiple source workspaces triggering one downstream
   → Cannot control order if multiple sources trigger simultaneously

5. Trigger respects workspace settings
   → If downstream has auto-apply: apply runs automatically
   → If downstream requires approval: apply waits for human
```

🔵 Short Interview Answer

> "Run triggers in TFC automate dependency management between workspaces. Configure them in the downstream workspace — 'trigger a plan when workspace X applies.' When the networking workspace applies, the compute workspace that depends on it automatically queues a plan to pick up any changes from networking outputs. This eliminates manual coordination between teams. Triggers fire on successful apply (not just plan), and only queue a plan — the downstream workspace still follows its own apply approval settings. For fan-out architectures, a single upstream workspace can trigger multiple downstream workspaces simultaneously, and a downstream workspace can require multiple upstream workspaces to have applied before triggering."

Topic 71: ➕ Cost Estimation in TFC — How It Works, Limitations

---

🔵 What It Is (Simple Terms)

Terraform Cloud's **cost estimation** feature calculates the estimated monthly cost of the resources in a Terraform plan — showing you what you'll spend before you apply. It appears as a step in the run lifecycle between plan and Sentinel policy checks.

🔵 How Cost Estimation Works

```
Run lifecycle with cost estimation:
  1. Plan runs
  2. Cost estimation evaluates the plan
     → Reads resource types and sizes from plan
     → Looks up pricing from AWS/GCP/Azure price lists
     → Calculates estimated monthly cost
  3. Shows cost summary in run output
  4. Sentinel policies can access cost data
  5. Apply proceeds (or waits for approval)

Cost estimation output:

   ┌──────────────────────────────────────────────────────────┐
   │ Cost Estimation                                          │
   │                                                          │
   │ Resources: 12 estimated, 3 unestimated                   │
   │                                                          │
   │ Cost changes:                                            │
   │   + aws_instance.web (t3.large)         $59.90/mo        │
   │   + aws_db_instance.main (db.r5.large)  $267.00/mo       │
   │   + aws_nat_gateway.main[0]             $32.40/mo        │
   │   ~ aws_lb.main (no change)             $0.00/mo         │
   │                                                          │
   │ Total monthly change: +$359.30                           │
   │ New estimated total: $1,247.80/mo                        │
   └──────────────────────────────────────────────────────────┘
```

🔵 Sentinel Integration — Cost-Based Policies

```
# sentinel/limit-plan-cost.sentinel
# Block applies that exceed cost thresholds

import "tfrun"

# Policy: plans that add more than $500/month need approval
monthly_budget_increase = 500.00

cost_within_budget = rule {
```

```
    tfrun.cost_estimate.delta_monthly_cost < monthly_budget_increase
}

main = rule {
    cost_within_budget
}

# With SOFT_MANDATORY: exceeding budget pauses apply for FinOps team
approval
# With HARD_MANDATORY: exceeding budget blocks apply entirely
```

```
# sentinel/require-cost-estimate.sentinel
# Ensure every run has cost estimation data

import "tfrun"

# Don't allow applies without cost estimation
main = rule {
    tfrun.cost_estimate.outcome is not "skipped"
}
```

🔵 Limitations of TFC Cost Estimation

```
What it DOES estimate:
  ✅ Common AWS: EC2, RDS, ELB, NAT Gateway, EKS, ElastiCache
  ✅ Common GCP: Compute Engine, Cloud SQL, GKE
  ✅ Common Azure: VMs, SQL Database, AKS

What it CANNOT estimate:
  ❌ Data transfer costs (too variable — depends on traffic)
  ❌ API call costs (Lambda invocations, DynamoDB requests)
  ❌ Storage I/O costs (EBS IOPS, RDS I/O)
  ❌ Third-party provider resources (Datadog, Snowflake, etc.)
  ❌ Custom resources
  ❌ Spot instance savings
  ❌ Reserved instance/savings plan discounts
  ❌ Actual usage-based services (S3 storage, CloudWatch logs)

Accuracy concerns:
  ⚠ Uses on-demand pricing — may significantly overestimate for
reserved/spot
  ⚠ Region-specific pricing varies — may use wrong region's price
  ⚠ Doesn't account for free tier
  ⚠ Does NOT include existing costs — only shows the delta

Real-world use:
  Cost estimation is a guardrail signal, not a budget tool
```

```
  Use for: catching accidental db.r5.8xlarge or 100 NAT gateways
  Use AWS Cost Explorer/Infracost for accurate budget planning
```

🔵 Infracost — Better Alternative for Cost Analysis

```
# Infracost: more accurate open-source cost estimation
pip install infracost
infracost configure set api_key <your-key>

# Generate cost breakdown
infracost breakdown --path .

# Show diff against main branch
infracost diff --path . --compare-to main

# Output: detailed per-resource cost breakdown with pricing sources
# Integrates with CI/CD as a separate step before terraform apply
# Supports 100+ resource types, reserved pricing, savings plans
```

🔵 Short Interview Answer

> "TFC cost estimation appears as a step between plan and Sentinel checks, showing estimated monthly cost delta for the resources in the plan. It's useful as a guardrail — catching someone accidentally specifying `db.r5.8xlarge` when they meant `db.t3.medium`. Sentinel policies can access cost data and block or require approval for applies that exceed budget thresholds. Key limitations: it uses on-demand pricing only (no reserved instance discounts), can't estimate data transfer, API calls, or usage-based costs, and shows the delta not the total existing cost. For accurate cost analysis, Infracost is a better tool — it supports more resource types, accounts for reserved pricing, and integrates directly into CI/CD pipelines."

Topic 72: ➕ `removed` Block (Terraform 1.7+) — Clean Resource Removal from State

---

🔵 What It Is (Simple Terms)

The `removed` block (Terraform 1.7+) is the declarative way to **remove a resource from Terraform management without destroying it**. It's the codified, Git-tracked alternative to `terraform state rm`.

🔵 The Problem It Solves

```
# Scenario: You managed an S3 bucket via Terraform
# You now want to manage it with a different tool (or stop managing it)
# But you don't want to DESTROY the bucket — just stop tracking it

# Old approach (terraform state rm):
terraform state rm aws_s3_bucket.legacy_data
# ✅ Works but:
# ❌ Not codified — not visible in config history
# ❌ Must be run manually on every environment
# ❌ Teammate doesn't know why the bucket disappeared from state
# ❌ Next terraform plan: shows bucket as "will be created" (confusing)

# New approach (removed block):
removed {
  from = aws_s3_bucket.legacy_data

  lifecycle {
    destroy = false   # remove from state only — don't destroy the real
bucket
  }
}
# ✅ Codified in config — PR reviewable, Git history traceable
# ✅ Applies to all environments automatically on next apply
# ✅ Self-documenting — clear intent in the codebase
# ✅ Plan clearly shows: "Will remove aws_s3_bucket.legacy_data from
state"
```

🔵 Full Syntax

```
# Basic: remove from state without destroying
removed {
  from = aws_s3_bucket.legacy_data

  lifecycle {
    destroy = false   # REQUIRED — must explicitly state whether to
destroy
  }
}

# Remove AND destroy the real resource
removed {
  from = aws_instance.old_server

  lifecycle {
    destroy = true   # plan shows: will destroy aws_instance.old_server
  }
}

# Remove an entire module from management
```

```
removed {
  from = module.deprecated_service

  lifecycle {
    destroy = false   # removes all resources in the module from state
  }
}

# Remove a count-indexed resource
removed {
  from = aws_instance.workers[2]

  lifecycle {
    destroy = false
  }
}

# Remove a for_each resource
removed {
  from = aws_instance.web["old-server"]

  lifecycle {
    destroy = false
  }
}
```

🔵 What Appears in Plan

```
terraform plan

# When destroy = false:
# Terraform will perform the following actions:
#
#   # aws_s3_bucket.legacy_data will no longer be managed by Terraform
#   # (destroy = false, bucket will be preserved)
#   - (removed)
#
# Plan: 0 to add, 0 to change, 1 to remove from state.

# When destroy = true:
# Terraform will perform the following actions:
#
#   # aws_instance.old_server will be destroyed
#   - resource "aws_instance" "old_server" {
#       - id            = "i-0abc123"
#       - instance_type = "t3.medium"
#     }
#
# Plan: 0 to add, 0 to change, 1 to destroy.
```

🔵 `removed` vs `terraform state rm` vs Deleting Config

```
Option 1: Delete config + run terraform apply (without removed block)
  Result: Terraform sees resource gone from config → plan shows "destroy"
  If you DON'T want to destroy: you must use removed or state rm first
  → Dangerous: accidentally deletes real infrastructure

Option 2: terraform state rm (imperative)
  Result: Resource removed from state, config still references it (until
you also delete config)
  ✅ Works immediately
  ❌ Not in Git history
  ❌ Must run manually per environment
  ❌ No plan review

Option 3: removed block with destroy = false (declarative — preferred)
  Result: Clean, reviewed, codified removal from state
  ✅ PR reviewable
  ✅ Automatic across all environments
  ✅ Plan shows what will happen
  ✅ Requires Terraform 1.7+

Decision:
  TF 1.7+, planned removal → removed block
  TF < 1.7, emergency removal → terraform state rm
  Remove AND destroy → removed block with destroy = true (or delete
config)
```

🔵 Combining `moved` and `removed`

```
# Common pattern: restructuring resources

# Old: managed as flat resources in root
# New: managed by a different tool (no Terraform involvement)

# Step 1: Stop managing the resource (removed block)
removed {
  from = aws_s3_bucket.audit_logs

  lifecycle {
    destroy = false   # DataOps team will manage this going forward
  }
}

# Another common combination:
# Move resource to new address, then remove from Terraform

moved {
```

```
  from = aws_instance.legacy
  to   = aws_instance.legacy_v2
}

# Next PR (after all envs have applied the moved block):
removed {
  from = aws_instance.legacy_v2
  lifecycle { destroy = false }
}
# ← Remove after confirming the moved block was applied everywhere
```

🔵 When to Remove `removed` Blocks

```
Same rule as moved blocks:
  Keep until ALL environments have applied
  Remove in a separate PR afterward

Environment checklist:
  ✅ dev has applied — removed block processed
  ✅ staging has applied — removed block processed
  ✅ prod has applied — removed block processed
  → NOW safe to remove the block from config

After removal: future plans won't mention the resource at all
  The resource is untracked — Terraform doesn't know it exists
```

🔵 Short Interview Answer

> "The `removed` block (Terraform 1.7+) is the declarative way to remove a resource from Terraform management without destroying it — the codified alternative to `terraform state rm`. You specify the resource address in `from` and set `lifecycle { destroy = false }` to preserve the real resource. On the next apply, Terraform removes the state entry and emits a clear plan showing what it will do. The key advantages over `state rm`: it's committed to Git (PR reviewable, self-documenting), automatically applies to all environments on next plan/apply, and leaves a clear audit trail of why the resource was unmanaged. Like `moved` blocks, keep `removed` blocks until all environments have applied, then clean them up."

📊 Category 9 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight

Topic	One-Line Summary	Interview Weight
62. CI/CD automation	Plan on PR (comment output), apply on merge, OIDC auth, separate roles	⭐⭐⭐⭐⭐
63. Atlantis vs TFC	Atlantis = self-hosted PR comments; TFC = SaaS + Sentinel + cost est	⭐⭐⭐⭐
64. Team workflows ⚠	Locking prevents concurrent corruption, plan artifact = apply what was reviewed	⭐⭐⭐⭐⭐
65. Environment promotion	Sequential pipeline: dev → test → staging → approval → prod	⭐⭐⭐⭐⭐
66. Terraform Cloud	VCS integration, remote execution, variable sets, run lifecycle	⭐⭐⭐⭐
67. Sentinel ⚠	advisory/soft-mandatory/hard-mandatory, policy sets, plan access	⭐⭐⭐⭐⭐
68. Useful flags	auto-approve (CI only), refresh=false (speed), parallelism (throttle)	⭐⭐⭐⭐
69. Agent pools	Agents in your VPC poll TFC for jobs — reach private resources	⭐⭐⭐
70. Run triggers	Upstream apply → downstream plan queued automatically	⭐⭐⭐
71. Cost estimation	Delta cost in run lifecycle, Sentinel cost policies, Infracost better	⭐⭐⭐
72. `removed` block	Declarative state rm (destroy=false), codified in Git, TF 1.7+	⭐⭐⭐⭐

🔑 Category 9 — Critical Rules

```
CI/CD pipeline:
  Plan role ≠ Apply role (read vs write permissions)
  Plan on PR, apply on merge — never the reverse
  Plan artifact: save with -out=tfplan, apply with apply tfplan
  Concurrency group in GitHub Actions prevents parallel applies

Plan artifacts:
  Saved plan = guarantee: what was reviewed = what was applied
  Plan artifacts expire — apply in same pipeline run when possible
  Upload as CI artifact, download in apply job

Terraform Cloud:
  Workspace = unit of isolation (state, variables, permissions, runs)
  Variable sets = shared config across workspaces (avoid duplication)
  Execution mode: remote (default), local, agent
  VCS integration: push → plan, merge → apply (or queued)

Sentinel:
```

```
    advisory: warning only, never blocks
    soft-mandatory: blocks, authorized users can override with reason
    hard-mandatory: blocks, NO override — absolute requirement
    Can access: plan data, cost estimates, tfrun metadata

  Flags:
    -auto-approve: CI/CD only, never manual prod applies
    -refresh=false: speed optimization, dangerous if drift possible
    -parallelism: doesn't affect dependent resources (only independent ones)
    -detailed-exitcode: 0=no changes, 1=error, 2=changes

  removed block (TF 1.7+):
    destroy = false: remove from state, preserve real resource
    destroy = true: remove from state AND destroy real resource
    Keep until all environments have applied, then clean up
```

🎯 Category 9 — Top 5 Interview Questions to Master

1. **"Walk me through your team's Terraform CI/CD pipeline"** — plan on PR (comment output, security scan, plan artifact), apply on merge (OIDC, separate apply role, approval gate, Slack notification)
2. **"What is a plan artifact and why does it matter?"** — `plan -out=tfplan` then `apply tfplan` guarantees what was reviewed = what was applied; prevents time-gap surprises
3. **"What's the difference between Atlantis and Terraform Cloud?"** — self-hosted vs SaaS, VPC access vs agents needed, PR comments vs UI, free vs paid, Sentinel built-in vs manual OPA
4. **"Explain Sentinel enforcement levels"** — advisory (warn only), soft-mandatory (block + override possible), hard-mandatory (block, no override) — match level to risk
5. **"When would you use `-refresh=false` and when is it dangerous?"** — use for speed when certain no drift; dangerous after incidents, in prod, or when manual changes may have occurred

---

> **Next:** Category 10 — Advanced Patterns, Testing & Troubleshooting (Topics 73–87) Type `Category 10` to continue, `quiz me` to be tested on Category 9, or `deeper` on any specific topic.